Synthesize Once, Repair Many: Amortizing LLM Costs in Automated Repair of Programming Assignments

Anonymous submission

Abstract

Large language models (LLMs) now repair most buggy submissions to introductory programming assignments, but they are deployed *one query per submission*: every request retransmits the assignment description and test suite before the model ever sees the student's code—in our measurements, 79% of prompt tokens are identical across submissions to the same assignment, and provider-side caching recovers only half of the resulting cost. We observe that assignment repair is a *many-instances, one-specification* regime, and propose to change the unit of LLM investment from the submission to the assignment. In CoEP, an LLM agent synthesizes—once per assignment—an executable symbolic repairer: problem-specific AST rewrite rules over a generic edit-search library, with every candidate patch validated against the test suite and capped in patch size. The repairer is refined for at most three feedback rounds on a handful of sample bugs, then deployed locally at zero marginal LLM cost, with conventional per-submission repair as fallback. On a contamination-free benchmark of 10 assignments and 229 buggy programs, with the same frontier model powering both paradigms, synthesized repairers fix 74.9% of 179 held-out submissions locally in $\sim$2s on a CPU; the hybrid matches the per-submission baseline's 100% repair rate and patch quality while cutting marginal cost $4.0\times$. The one-off build cost ($0.126 per assignment) breaks even after ≈ 56 submissions—well within a single course offering. Ablations locate the power: generic edit search alone fixes 95% of mutation-style bugs but only 21% of misconception bugs, which the synthesized assignment-specific rules lift to 50% ($p<10^{-6}$). Amortizing LLM reasoning into verifiable symbolic artifacts is, we argue, a general recipe for AI systems that must serve many instances of one specification.

Introduction

Timely, individualized feedback is the scarcest resource in programming education. Introductory courses routinely enroll hundreds of students on campus and tens of thousands online, each producing a stream of buggy submissions that instructors cannot examine one by one. Automated program repair (APR) has therefore long been proposed as a feedback engine for programming assignments: given a buggy student submission and the assignment's test suite, produce a small patch that makes the submission correct while preserving the student's own approach (Yi et al. 2017; Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022).

Large language models (LLMs) have made this vision practical. LLM-based repair now fixes the large majority of real student programs—up to 96.5% of buggy Python submissions in PyDex (Zhang et al. 2024)—and general-purpose LLM repair systems report strong results at modest per-bug prices (Xia and Zhang 2024, 2022; Liu et al. 2024; Yang et al. 2024). But these systems all share a deployment paradigm that we argue is mismatched with the economics of the classroom: *every buggy submission triggers a fresh LLM query* that re-transmits the assignment description, the test suite, and formatting instructions before the model ever sees the student's code. In the benchmark we study, 79% of the input tokens of a per-submission repair prompt are byte-identical across all submissions to the same assignment; the submission itself is a small minority of what the course pays to process, query after query, semester after semester. Provider-side prompt caching (Anthropic 2025; OpenAI 2024) discounts the redundant prefix but does not change the paradigm: in our measurements it removes 49% of the per-submission cost, and every submission still requires a round trip to a frontier model, with the attendant marginal cost, latency, rate limits, and dependence on an external service.

Our key observation is that assignment repair is a *many-instances, one-specification* regime. A single assignment is attempted by hundreds or thousands of students whose failures cluster into a modest number of recurring misconceptions (Gulwani, Radicek, and Zuleger 2018; Hu et al. 2019). Pre-LLM assignment-repair systems exploited exactly this structure—CLARA clustered correct solutions, Refactory refactored them into a per-problem search space—achieving cheap per-submission repair, but at the cost of hand-built machinery or large pools of prior correct solutions. Per-submission LLM repair inverts the trade-off: it is automatic and general, but pays for the same specification-level reasoning on every single query.

We propose to get both halves: **spend the LLM once per assignment, not once per submission**. In our system, CoEP (Cost-Effective Program repair), an LLM agent is given the assignment specification, its test suite, a few reference solutions, and a handful of historical buggy submissions, and is asked to *write a program repairer*: a determin-

istic Python module that combines problem-specific rewrite rules with a generic edit-search library, validates every candidate against the visible tests, and enforces a patch-size cap so that repairs remain minimal edits of the student's code. The synthesized repairer is refined for at most three feedback rounds against the sample bugs, then frozen and deployed: each incoming submission is repaired *locally*, in seconds, at zero marginal LLM cost. Submissions the local repairer cannot fix escalate to the conventional per-submission LLM path, so the hybrid system never does worse than the paradigm it replaces.

On a benchmark of 10 introductory Python assignments with 229 buggy programs (misconception-crafted and mutation-generated), holding the underlying LLM fixed across all conditions, CoEP's synthesized repairers alone fix 74.9% of 179 held-out submissions; with LLM fallback the hybrid fixes 100.0%—exactly matching the per-submission frontier-LLM baseline in both repair rate and patch quality, with minimality now guaranteed by a validator rather than requested in a prompt. The one-off synthesis cost of $0.126 per assignment is recouped after ≈56 repaired submissions, and thereafter each repair costs 4.0× less than per-submission prompting—a gap that no amount of prompt caching closes, because 25.1% is the only fraction of traffic that touches the LLM at all.

This paper makes the following contributions:

- **A new deployment paradigm for assignment repair.** We identify the many-instances, one-specification structure of programming-assignment APR and formulate *per-assignment repairer synthesis*: amortizing LLM reasoning into a reusable, locally executable repair artifact, with a cost model and break-even analysis for when it pays off.
- **The CoEP system.** A builder agent that synthesizes per-assignment repairers through a generate–validate–refine loop (Madaan et al. 2023; Shinn et al. 2023) over a repair-operator library; repairers are independently re-validated (test-suite pass + relative-patch-size cap) so a repairer can never self-report success, and are backed by hybrid LLM escalation.
- **A controlled, contamination-free evaluation.** A new benchmark of 10 assignments / 229 buggy programs built from a misconception taxonomy plus seeded mutation (Just et al. 2014), with both repair paradigms driven by the *same* frontier model, exact token-level cost accounting, and statistical tests; plus ablations isolating the contribution of synthesized problem-specific rules and of refinement rounds.
- **Artifacts.** We release the benchmark, harness, prompts, synthesized repairers, and all raw model outputs.[1]

The Cost Anatomy of Per-Submission Repair

To make the redundancy argument concrete, Table 1 decomposes the per-submission repair prompt used by our baseline (and, mutatis mutandis, by PyDex- and ChatRepair-style systems (Zhang et al. 2024; Xia and Zhang 2024)): a task preamble, the assignment description, the visible test suite, minimal-patch instructions, and finally the buggy submission. Averaged over our benchmark, a single repair query consumes 740 input and 51 output tokens, of which only ≈21% of the input varies between submissions to the same assignment.

Prompt component	Tokens (mean)	Share
Task preamble + instructions Assignment description Visible test suite	583	79%
Buggy submission (varies)	157	21%
Total input / query	740	
Output / query	51	

Table 1: Cost anatomy of a per-submission repair query, averaged over 179 queries. The first three components are byte-identical for every submission to the same assignment; the paradigm re-buys them on every query.

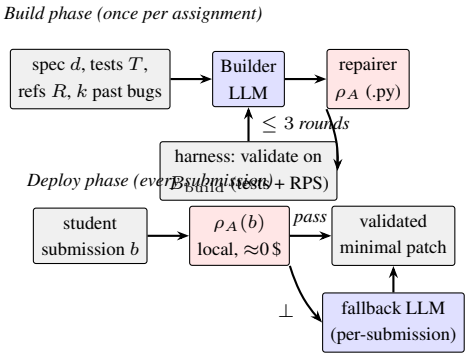


Figure 1: CoEP. LLM effort (blue) is spent once per assignment to synthesize a local repairer (red); at deployment, most submissions never touch a model.

Three consequences follow. (1) *Caching is a discount, not a fix.* Prompt caching bills the shared prefix at 0.1× after a 1.25× write (Anthropic 2025), cutting per-submission cost from $\$2.99 \times 10^{-3}$ to $\$1.53 \times 10^{-3}$ in our setting (-49%)—but the submission-dependent tokens, all output tokens, and one frontier-model inference per submission remain. (2) *Costs scale with enrollment, not with assignments.* A department running 40 course sections, each with hundreds of buggy submissions per assignment, pays per student-attempt; the specification-level reasoning it is paying for is identical every time. (3) *The redundancy is exactly the amortizable part.* What is constant per assignment—understanding the specification, the tests, and the space of likely mistakes—can be reified *once* into an executable artifact. That is the move CoEP makes, and the rest of the paper quantifies what it buys.

[1] Anonymized repository submitted as supplementary material.

Approach

Problem Formulation

An *assignment* is a triple $A = (d, T, R)$: a natural-language description d, a visible test suite $T = \{(x_i, y_i)\}$ over a designated entry function, and a small set R of reference solutions. A *repair* of a buggy submission b is a program p such that (i) p passes all of T and (ii) $\mathrm{RPS}(b, p) \le \tau$, where RPS is the token-level relative patch size—the Levenshtein distance between the lexical token sequences of b and p, normalized by $|b|$—following the patch-size metrics of Refactory and PyDex (Hu et al. 2019; Zhang et al. 2024). The cap τ (we use $\tau = 0.35$) operationalizes the pedagogical requirement that a repair be feedback on the *student's* program: it rules out replacing the submission with a reference solution, which passes tests but teaches nothing (Yi et al. 2017).

Conventional LLM-based APR maps each b to a query $\mathrm{LLM}(d, T, b)$. CoEP instead synthesizes, once per assignment, a *repairer* $\rho_A : b \mapsto p \cup \{\perp\}$—an ordinary Python module with no model access—and deploys the hybrid

$$\mathrm{repair}(b) = \begin{cases} \rho_A(b) & \text{if } \rho_A(b) \neq \perp \ (\approx 0 \text{ cost}) \\ \mathrm{LLM}(d, T, b) & \text{otherwise.} \end{cases}$$

Repairer Synthesis

A builder agent receives d, T, R, and k=5 historical buggy submissions B_{build}, together with the documentation of a small *repair-operator library* that we provide as the substrate for all synthesized repairers. The library contributes three reusable capabilities:

(1) Generic edit search. `candidates_generic` enumerates candidate patches by applying small semantic edits at every applicable AST site—comparison/arithmetic/Boolean operator swaps, integer off-by-one shifts, Boolean-constant flips, index shifts, and argument swaps—singly or in pairs. This is the inversion of the classical mutation-operator space (Just et al. 2014) and plays the role that hand-curated fix templates play in template-based APR (Kim et al. 2013; Liu et al. 2019).

(2) Test-driven validation. `first_passing` runs candidates against the full suite T in a sandbox and returns the first candidate that passes *all* tests while satisfying the RPS cap. Synthesized code is contractually forbidden from returning an unvalidated patch.

(3) Failure diagnosis. `failing_tests` exposes the failing-test pattern of the input program, letting repairers dispatch: e.g., a `flatten` submission whose output is correct but reversed selects stack-order rules before anything else.

The builder's essential creative work is the *problem-specific rule set*: AST rewrite rules that anticipate the misconceptions students exhibit for *this* assignment—wrong recursion base cases for `fibonacci`, count-only bracket matching for `balanced_brackets`, order-destroying `set()` deduplication for `unique_in_order`—derived by studying d, T, R, and B_{build}. These rules generate edits far outside the generic operator space (inserting a missing `else` branch, wrapping a loop body in a recursive call), and our ablation shows they, not the generic search, carry the paradigm (RQ3).

Algorithm 1 Per-assignment repairer synthesis

Require: assignment (d, T, R), sample bugs B_{build}, rounds $R_{\max}$

$\rho \leftarrow \mathrm{LLM}_{\mathrm{build}}(d, T, R, B_{\mathrm{build}}, \mathrm{lib})$ ▷ single shot
for $r = 2 \ldots R_{\max}$ **do**
  $F \leftarrow \{(b, \mathrm{outcome}(\rho, b)) : b \in B_{\mathrm{build}}\}$ ▷ harness
  if all outcomes are REPAIRED **then**
    return ρ
  end if
  $\rho \leftarrow \mathrm{LLM}_{\mathrm{refine}}(\rho, F)$ ▷ single shot
end for
return ρ

Generate–validate–refine. Because a single-shot synthesized module can contain its own defects, the harness closes the loop (Algorithm 1): it installs the module, applies it to B_{build}, *independently* re-validates every returned patch (sandboxed test execution plus RPS check—the repairer's own claims are never trusted), and, if any sample bug is unrepaired, sends the module back with per-bug outcomes and the failing submissions for revision, up to $R_{\max}$=3 total rounds. This is a deliberately austere instantiation of self-refinement (Madaan et al. 2023; Shinn et al. 2023): every LLM interaction is a single prompt–response pair, so the entire build cost is the token count of at most $2R_{\max}$ message payloads, measured exactly.

Deployment. The frozen repairer runs in a subprocess with a wall-clock budget (60s; observed mean 2.1s) and no network access. Its patches are re-validated by the grading harness before being shown to a student, so a defective repairer degrades to $\perp$ (escalation), never to a wrong "repair." Escalated submissions follow the conventional per-submission LLM path; their cost is counted against CoEP in every comparison.

Why synthesis beats retrieval. An alternative amortization is to cache past LLM *patches* and retrieve them for similar submissions. But a patch is specific to one buggy program, whereas a synthesized rule generalizes over a misconception class: the repairer fixes submissions whose surface form it has never seen, which pure retrieval cannot (we return to this in Related Work).

Experimental Setup

We ask three research questions: **RQ1 (Effectiveness & quality).** How do CoEP's synthesized repairers compare to per-submission frontier-LLM repair in repair rate and patch quality? **RQ2 (Cost).** What are the token/dollar costs of both paradigms, when does repairer synthesis break even, and what is the asymptotic saving? **RQ3 (Sources of power).** How much do the synthesized problem-specific rules and the refinement rounds contribute?

Benchmark

Public assignment-repair datasets (ITSP, the Refactory dataset, TutorCode) have circulated on GitHub for years and are plausibly in the training data of every current frontier model; memorization would inflate *both* paradigms' re-

pair rates and specifically contaminate the builder, which must not have seen the evaluation bugs (Yang et al. 2024). Following the data-leakage guidance for LLM evaluations in APR (Xia and Zhang 2024), we therefore constructed a fresh benchmark whose held-out buggy programs cannot have been memorized, while borrowing its *design* from established suites.

The benchmark comprises 10 classic introductory Python assignments (digit sum, palindrome check, Fibonacci, consecutive deduplication, top-k selection, sorted insertion index, bracket matching, GCD, list flattening, grade statistics), each with an assignment handout, 14 visible tests including edge cases, and three reference solutions implementing genuinely different strategies. Each assignment has two pools of buggy programs: **(a) Misconception-crafted** (10 per assignment): realistic novice attempts written to instantiate documented misconception categories—wrong base cases, off-by-one boundaries, wrong initialization, order-destroying deduplication, count-only bracket matching, single-level flattening, integer-division means—in the spirit of the qualitative bug analyses of CLARA and Refactory (Gulwani, Radicek, and Zuleger 2018; Hu et al. 2019), with structural diversity across implementations. **(b) Mutation-generated** (up to 15 per assignment): seeded first- and second-order mutants of the reference solutions under classical mutation operators, the standard proxy for real faults (Just et al. 2014) used by IntroClass, Codeflaws, and QuixBugs (Le Goues et al. 2015; Tan et al. 2017; Lin et al. 2017). Every buggy program is machine-checked to compile and fail at least one test; every reference passes all tests. In total: 229 buggy programs; per assignment, 5 (2 crafted + 3 mutants) form the builder's B_{build} and up to 20 are held out for evaluation (n=179). The crafted/mutant distinction doubles as an evaluation instrument: mutants lie inside the generic operator space by construction, so *crafted bugs are where problem-specific reasoning must earn its keep* (RQ3).

Baselines

Per-submission LLM repair is the deployment paradigm of PyDex/MMAPR, ChatRepair, and FastFixer (Zhang et al. 2024, 2022; Xia and Zhang 2024; Liu et al. 2024): one query per submission containing the handout, the visible tests, the buggy program, and minimal-patch instructions (the strongest known configuration for student programs; PyDex reports up to 96.5% with few-shot prompting). We instantiate it with the *same* frontier model that powers CoEP's builder, so every observed difference is attributable to the paradigm, not the model. **Cached per-submission repair** re-prices the same queries under provider prompt caching (shared prefix written once at $1.25\times$, read at $0.1\times$) (Anthropic 2025)—the strongest cost optimization available *within* the per-submission paradigm. We do not additionally re-run pre-LLM per-problem tools (Refactory, CLARA) as repair-rate baselines: they require dozens-to-hundreds of prior correct solutions per assignment and are consistently dominated by LLM repair on student programs (Zhang et al. 2024); our contribution is orthogonal to the choice of local technique.

	Repair rate	RPS$\leq$0.35	RPS	\$/sub
LLM per-subm.	**100.0** [97.9, 100]	92.7	0.127	2.99×10^{-3}
+ caching	**100.0** [97.9, 100]	92.7	0.127	1.53×10^{-3}
Generic-only	62.0 [54.7, 68.8]	62.0	0.043	0
CoEP-local	74.9 [68.0, 80.6]	74.9	0.058	**0**
CoEP-hybrid	**100.0** [97.9, 100]	92.7	0.127	0.75×10^{-3}

Table 2: Main results on 179 held-out buggy submissions (95% Wilson CIs). "RPS$\leq$0.35" counts only repairs within the patch-size cap; local methods enforce it by construction. Marginal cost excludes the one-off \$0.126/assignment build cost, analyzed in RQ2. ($\times10^{-3} = \times10^{-3}$\$.)

Metrics

Repair rate: fraction of evaluation submissions for which the system produces a program passing all visible tests (Hu et al. 2019; Zhang et al. 2024). For the baseline we additionally report the *quality repair rate*, counting only repairs with $\text{RPS} \leq 0.35$—the constraint CoEP enforces by construction—since a test-passing rewrite that discards the student's approach is not usable feedback (Yi et al. 2017). **Relative patch size (RPS)**: token-level edit distance normalized by program length (Hu et al. 2019; Zhang et al. 2024); smaller is pedagogically better. **Cost**: exact token counts of every prompt/response pair, priced at a representative frontier tier (\$3/\$15 per Mtok in/out; Claude Sonnet standard pricing, July 2026) with cache pricing as above, reported as tokens, USD per submission, and USD per assignment, following the per-bug cost accounting of ChatRepair (Xia and Zhang 2024). Repair rates carry 95% Wilson intervals; paired comparisons use exact McNemar tests.

Protocol

All LLM interactions—179 baseline repairs and every builder/refinement call—are single-shot prompt→response exchanges executed by the same frontier model (Claude, July 2026 snapshot) under identical instructions, with no code execution or test access at generation time; all validation is performed by our harness. Local search is deterministic; LLM calls are single-sample (n=1), matching the deployment setting of one repair attempt per submission. Costs are computed from the exact message strings with a fixed tokenizer approximation applied identically to both arms (chars/3.6; provider tokenizers are unavailable offline), so all cost *ratios* are tokenizer-independent. Experiments ran on a single Linux container (no GPU); synthesized repairers used a 60s timeout and τ=0.35 throughout.

Results

RQ1: Effectiveness and Patch Quality

Table 2 presents the headline comparison. The per-submission frontier-LLM baseline repairs 100.0% of submissions—consistent with the near-saturated rates modern LLMs achieve on introductory assignments (Zhang et al. 2024)—which makes it a demanding reference point: the question this paper poses is not whether LLMs can repair

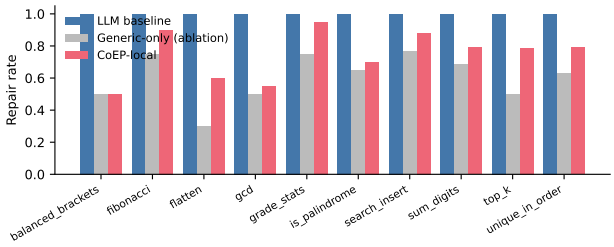


Figure 2: Per-assignment repair rates. The gap between generic-only (grey) and CoEP-local (red) is the contribution of synthesized problem-specific rules.

student code, but what each repair needs to cost. CoEP's synthesized repairers alone fix 74.9% (CI [68.0, 80.6]) at zero marginal LLM cost, in 2.1s per submission on a CPU; with fallback the hybrid reaches 100.0%, exactly matching the baseline, as guaranteed by construction.

Patch quality is preserved, and its *guarantee* changes character. The baseline model may rewrite freely, and does: 7.3% of its test-passing "repairs" (13/179) exceed the RPS cap, discarding the student's approach—the failure mode Yi et al. (2017) identified as pedagogically useless. CoEP cannot produce such a patch: cap compliance is checked mechanically on every candidate. A paired view shows there is no hidden quality tax: on the 134 submissions CoEP repairs locally, its patches are exactly as minimal as the baseline's on the same instances (mean RPS 0.058 for both); the baseline's heavier aggregate tail (mean 0.127, driven by the 13 rewrites) falls entirely on submissions CoEP escalates anyway, so the hybrid's quality profile (92.7% cap-compliant) equals the baseline's. What changes is enforcement: minimality and determinism hold for local repairs *by construction*—a prompt can request these properties, but only a validator can guarantee them.

Per-assignment rates (Figure 2) range from 50% (`balanced_brackets`, whose stack-discipline bugs often need multi-edit structural fixes) to 95% (`grade_stats`). The dispersion tracks how far an assignment's misconceptions lie from small-edit repairs—a property instructors can estimate in advance, and a natural signal for routing hard assignments to a larger build budget.

RQ1. Synthesized repairers fix 74.9% of held-out submissions locally; the hybrid matches the frontier-LLM baseline exactly in repair rate (100.0%) and patch quality (92.7% cap-compliant; equal RPS on shared instances)—while making minimality and determinism guarantees the per-submission paradigm can only request.

RQ2: Cost and Break-Even

Table 3 gives the exact accounting. Synthesizing an assignment's repairer costs $0.126 on average (47493 prompt + 74407 response tokens across all ten assignments)—the price of roughly 42 baseline repairs, paid once. Let C be the build cost, c the per-submission LLM cost, and f the fallback fraction. CoEP is cheaper than per-submission repair

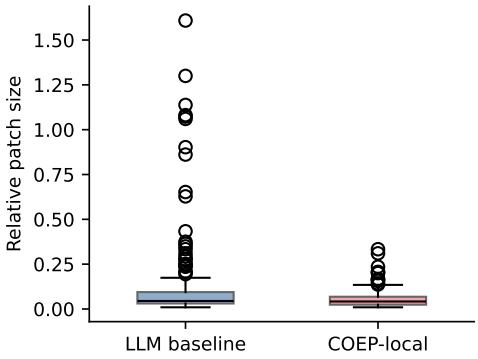


Figure 3: Relative patch size of successful repairs. CoEP patches are minimal by construction; the baseline's distribution has a heavy rewrite tail.

	Tokens (in / out)	USD
Baseline, per submission	740 / 51	2.99×10^{-3}
with prompt caching	740 / 51	1.53×10^{-3}
CoEP build, per assignment	4749 / 7441	126×10^{-3}
CoEP marginal, per submission	0 / 0	0
CoEP fallback, per escalation	739 / 54	3.02×10^{-3}

Table 3: Token and dollar accounting ($\times10^{-3}$ = $\times10^{-3}$\$). Build cost covers all builder and refinement calls (15 LLM calls across 10 assignments, $R_{\max}$=3).

once

$$N > N^* = \frac{C}{c\,(1-f)},$$

which with measured values (C=\$0.126, c=\$2.99 $\times$ 10^{-3}, f=0.251) gives $N^* \approx 56$ submissions per assignment—about one week of a mid-sized course section, and an order of magnitude below the ≈340 buggy submissions per assignment in the Refactory corpus (Hu et al. 2019). Against the prompt-cached baseline, $N^* \approx 110$. Asymptotically each submission costs $f \cdot c$: a 4.0× reduction over per-submission prompting, and—because caching applies equally to the fallback traffic—the same factor over the cached baseline. Prompt caching, the strongest optimization *within* the pay-per-query paradigm, saves 49%; changing the paradigm saves 75%, and the two compose.

Figure 4 plots cumulative cost per assignment. Beyond cost, the deployment profile changes qualitatively: 74.9% of submissions are repaired in 2.1s on a CPU with no network dependency, no rate limits, and deterministic, auditable behavior—operational properties that per-query paradigms cannot offer at any price.

RQ2. A \$0.126 one-off build per assignment breaks even after ≈56 submissions and cuts marginal repair cost 4.0×; provider-side caching saves 49% within the old paradigm, while CoEP composes with it and removes 75% of queries outright.

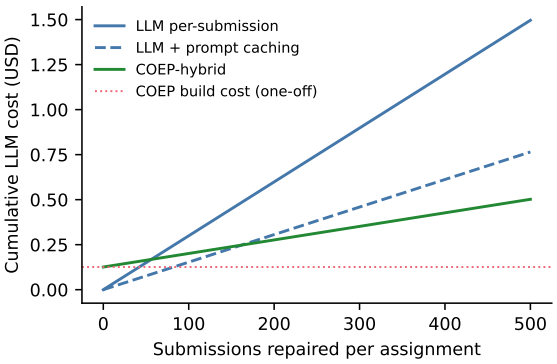


Figure 4: Cumulative LLM cost per assignment as submissions accumulate. CoEP's one-off build cost (dotted) is overtaken at $N^* \approx 56$ submissions (vs. the uncached baseline); its slope thereafter is $f \cdot c$, a 4.0× reduction.

RQ3: Where Does the Power Come From?

Synthesized rules vs. generic search. The generic-only ablation—the same operator library and validation, with no LLM-synthesized rules—repairs 62.0% overall, but the aggregate hides the mechanism. On mutation-generated bugs both variants fix 94/99 (95%): mutants lie inside the generic operator space by construction, so inverting them requires no assignment understanding. On misconception-crafted bugs the generic search collapses to 17/80 (21%) while full CoEP reaches 40/80 (50%)—the synthesized rules *more than double* repair rate exactly where bugs stop being single-token slips (exact McNemar, discordant 23:0, $p{=}2.4 \times 10^{-7}$). Qualitatively, the rules that fire on crafted bugs are structural: inserting a missing `else` branch, wrapping a loop body in a recursive call, replacing order-destroying `set()` deduplication with an adjacency comparison—edits no generic mutation operator produces. This is the paradigm's core claim made measurable: what the builder amortizes is assignment-specific *understanding*, not search.

Refinement rounds. Seven of ten builders produced a repairer that fixed all five build samples in a single shot; three did not (two returned no patches at all due to internal defects, one missed structural cases). Feedback rounds repaired the builders themselves: round 2 fixed `is_palindrome` outright ($0/5 \rightarrow 5/5$; its eventual held-out rate, 70%, is indistinguishable from single-shot builders), and rounds 2–3 lifted `flatten` and `unique_in_order` from 0–$3/5$ to $4/5$. Overall build-sample success rose from $38/50$ (round 1) to $48/50$ (final)—evidence that the validate-and-refine loop, not single-shot generation luck, makes synthesis reliable; feedback that includes the still-failing submissions (added in round 3) was what unlocked the two hardest builders.

RQ3. Generic edit search suffices only for mutation-style slips (95%); on misconception bugs, LLM-synthesized assignment-specific rules lift local repair from 21% to 50% ($p{<}10^{-6}$). Harness-driven refinement turns unreliable single-shot builders (7/10 usable) into reliable ones (10/10).

Discussion and Limitations

Construct validity. Repair rate is measured against visible tests, as in prior assignment-repair work (Hu et al. 2019; Zhang et al. 2024); a patch could overfit the suite. Two design choices bound this risk: suites include adversarial edge cases (14 tests per assignment), and the RPS cap prevents the degenerate overfit of wholesale replacement—a failure mode we in fact observe in the *baseline*, 7.3% of whose "repairs" exceed the cap. Our cost model prices one provider tier; because we report raw token counts, any pricing can be substituted, and the break-even expression in RQ2 is closed-form in the price.

Internal validity. Both paradigms use the same model, the same single-attempt budget, and the same independent validation harness, and the builder never sees evaluation bugs. The LLM calls were executed through an agent harness constrained to behave as a stateless single-shot endpoint (no tool use, no test execution); reported costs are computed from the exact message payloads. Token counts use a fixed approximate tokenizer applied to both arms; ratios and break-even points are therefore unaffected by calibration error, and we release all raw strings for exact re-tokenization.

External validity. Our benchmark is synthetic-but-principled: crafted misconceptions plus seeded mutants, not real student submissions—the price of a contamination-free comparison. Real distributions are heavier-tailed: more duplicates and near-duplicates (which *favor* amortization—a rule pays off more often) but also messier failure modes (I/O handling, timeouts, partial attempts) that will lower local repair rates and raise the fallback fraction f; the hybrid design degrades gracefully in exactly that case, and the break-even analysis makes the dependence on f explicit. We evaluate one language (Python), one model family, and assignments of introductory scale; per-assignment synthesis for multi-function projects is open. Finally, our headline economics assume assignments recur across offerings—true of the CS1 canon by design, and the regime (hundreds of submissions per assignment) matches public corpora such as the Refactory dataset (≈340 buggy submissions per assignment) (Hu et al. 2019).

Related Work

APR for programming assignments. Pre-LLM systems exploit per-assignment structure: AutoGrader-style error models, CLARA's clustering of correct solutions (Gulwani, Radicek, and Zuleger 2018), Sarfgen's aligned-example search (Wang, Singh, and Su 2018), Refactory's refactoring-based online repair (Hu et al. 2019), and Verifix's verified alignment (Ahmed et al. 2022); Yi et al. (2017) established test-based repair rates and the feedback-suitability concern that motivates patch-size metrics. These systems are

cheap per submission but need substantial per-assignment inputs (many correct solutions, alignment machinery). At AAAI, assignment repair remains active: counterexample-guided repair with MaxSAT fault localization (Orvalho, Janota, and Manquinho 2025) and minimal-modification adaptive repair (Dai et al. 2025) both target exactly our setting's concerns (instructor burden, patch minimality) from the per-submission side. CoEP keeps the classical systems' per-assignment amortization but replaces their hand-built machinery with LLM synthesis from a handful of examples.

LLM-based repair. AlphaRepair, RING, and successors recast repair as generation (Xia and Zhang 2022); ChatRepair adds conversational test feedback and per-bug cost accounting (Xia and Zhang 2024); PyDex/MMAPR target student programs specifically (Zhang et al. 2024, 2022); FastFixer accelerates per-query inference for assignments (Liu et al. 2024); CREF studies tutor-in-the-loop conversation (Yang et al. 2024). All are per-submission: each buggy program pays for model inference. CoEP is complementary—any of them can serve as its fallback path—but shifts the primary cost from per-submission to per-assignment.

Template- and pattern-based APR. PAR and TBar showed that a fixed library of human-curated fix templates repairs a surprising share of real bugs (Kim et al. 2013; Liu et al. 2019). CoEP's generic operator layer is a mutation-inversion instance of this idea (Just et al. 2014); its novelty is the *synthesized, assignment-specific* template layer, authored per specification by an LLM rather than curated once globally by humans.

Cost-efficient LLM inference. FrugalGPT's cascades route queries between models of different prices (Chen, Zaharia, and Zou 2024); SRepair engineers cheap open-model pipelines ($0.029/bug) (Xiang et al. 2024); provider prompt caching discounts repeated prefixes (Anthropic 2025; OpenAI 2024). All optimize *within* the pay-per-query paradigm and are composable with CoEP (our cached baseline; a cheaper fallback model). CoEP instead removes queries: the majority of submissions never reach a model.

LLMs synthesizing reusable tools. Voyager's skill library and LLMs-as-tool-makers show agents that write reusable code artifacts amortized across future episodes (Wang et al. 2024; Cai et al. 2024), and self-refinement loops improve one-shot generations (Madaan et al. 2023; Shinn et al. 2023). CoEP carries this idea into APR with two twists that the education setting forces: the synthesized tool must be *verifiable* (test-suite validation, never-trust-self-reports) and *quality-constrained* (RPS cap), because its output is feedback to a learner.

Conclusion

Programming-assignment repair has a structure that per-submission LLM querying ignores and per-assignment synthesis exploits: one specification, many failures, recurring misconceptions. CoEP turns cents of frontier-model reasoning into a permanent, local, verifiable repairer per assignment—matching the per-submission paradigm's repair rate and patch quality at 4.0× lower marginal cost—and its break-even point sits well inside a single course offering. We see per-assignment synthesis as one instance of a broader recipe—*amortize LLM reasoning into executable artifacts wherever many instances share a specification*—that applies to grading rubrics, hint generation, and beyond.

Ethical Statement

CoEP automates feedback on student code; it does not automate grading decisions, and we advocate deploying it as a hint/feedback engine with instructor oversight. Two risks deserve note. *Feedback fairness*: a synthesized repairer's rule set reflects the sample bugs it was built from; if those samples underrepresent some students' approaches, local repair rates may differ across approaches. The hybrid design mitigates this—submissions the repairer cannot handle receive full-LLM treatment rather than degraded service—and per-approach repair-rate auditing is cheap because local repair is deterministic and inspectable. *Learning displacement*: instant repairs can short-circuit productive struggle; the RPS cap keeps output in the form of minimal, explainable edits rather than finished solutions, and deployments can surface failing tests before patches. Our benchmark contains no student data; all buggy programs are authored or generated for this study, avoiding the consent and privacy issues of real submission corpora.

References

Ahmed, U. Z.; Fan, Z.; Yi, J.; Al-Bataineh, O. I.; and Roychoudhury, A. 2022. Verifix: Verified Repair of Programming Assignments. *ACM Transactions on Software Engineering and Methodology*, 31(4).

Anthropic. 2025. Prompt Caching — Claude API Documentation. https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching. Cache reads are billed at 0.1x the base input-token price; accessed July 2026.

Cai, T.; Wang, X.; Ma, T.; Chen, X.; and Zhou, D. 2024. Large Language Models as Tool Makers. In *The Twelfth International Conference on Learning Representations, ICLR 2024*.

Chen, L.; Zaharia, M.; and Zou, J. 2024. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research*.

Dai, Z.; Chen, B.; Zhao, Z.; Tang, X.; Wu, S.; Yao, C.; Gao, Z.; and Chen, J. 2025. Less is More: Adaptive Program Repair with Bug Localization and Preference Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Gulwani, S.; Radicek, I.; and Zuleger, F. 2018. Automated Clustering and Program Repair for Introductory Programming Assignments. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*, 465–480. ACM.

Hu, Y.; Ahmed, U. Z.; Mechtaev, S.; Leong, B.; and Roychoudhury, A. 2019. Re-Factoring Based Program Repair Applied to Programming Assignments. In *34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019*. IEEE.

Just, R.; Jalali, D.; Inozemtseva, L.; Ernst, M. D.; Holmes, R.; and Fraser, G. 2014. Are Mutants a Valid Substitute for

Real Faults in Software Testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2014*. ACM.

Kim, D.; Nam, J.; Song, J.; and Kim, S. 2013. Automatic Patch Generation Learned from Human-Written Patches. In *Proceedings of the 35th International Conference on Software Engineering, ICSE 2013*. IEEE.

Le Goues, C.; Holtschulte, N.; Smith, E. K.; Brun, Y.; Devanbu, P. T.; Forrest, S.; and Weimer, W. 2015. The ManyBugs and IntroClass Benchmarks for Automated Repair of C Programs. *IEEE Transactions on Software Engineering*, 41(12): 1236–1256.

Lin, D.; Koppel, J.; Chen, A.; and Solar-Lezama, A. 2017. QuixBugs: A Multi-Lingual Program Repair Benchmark Set Based on the Quixey Challenge. In *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity, SPLASH Companion 2017*, 55–56. ACM.

Liu, F.; Liu, Z.; Zhao, Q.; Jiang, J.; Zhang, L.; Li, G.; Sun, Z.; Li, Z.; and Ma, Y. 2024. FastFixer: An Efficient and Effective Approach for Repairing Programming Assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024*. ACM.

Liu, K.; Koyuncu, A.; Kim, D.; and Bissyandé, T. F. 2019. TBar: Revisiting Template-Based Automated Program Repair. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019*, 31–42. ACM.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

OpenAI. 2024. Prompt Caching — OpenAI API Documentation. https://platform.openai.com/docs/guides/prompt-caching. Automatic prefix caching with discounted cached input tokens; see also https://openai.com/index/api-prompt-caching/; accessed July 2026.

Orvalho, P.; Janota, M.; and Manquinho, V. 2025. Counterexample Guided Program Repair Using Zero-Shot Learning and MaxSAT-based Fault Localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

Tan, S. H.; Yi, J.; Yulis; Mechtaev, S.; and Roychoudhury, A. 2017. Codeflaws: A Programming Competition Benchmark for Evaluating Automated Program Repair Tools. In *39th IEEE/ACM International Conference on Software Engineering, ICSE 2017 – Companion Volume*, 180–182. IEEE.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*. ArXiv:2305.16291.

Wang, K.; Singh, R.; and Su, Z. 2018. Search, Align, and Repair: Data-Driven Feedback Generation for Introductory Programming Exercises. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*. ACM.

Xia, C. S.; and Zhang, L. 2022. Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022*. ACM.

Xia, C. S.; and Zhang, L. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, 819–831. ACM.

Xiang, J.; Xu, X.; Kong, F.; Wu, M.; Zhang, Z.; Zhang, H.; and Zhang, Y. 2024. How Far Can We Go with Practical Function-Level Program Repair? arXiv preprint arXiv:2404.12833.

Yang, B.; Tian, H.; Pian, W.; Yu, H.; Wang, H.; Klein, J.; Bissyandé, T. F.; and Jin, S. 2024. CREF: An LLM-Based Conversational Software Repair Framework for Programming Tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, 882–894. ACM.

Yi, J.; Ahmed, U. Z.; Karkare, A.; Tan, S. H.; and Roychoudhury, A. 2017. A Feasibility Study of Using Automated Program Repair for Introductory Programming Assignments. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2017*, 740–751. ACM.

Zhang, J.; Cambronero, J.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2022. Repairing Bugs in Python Assignments Using Large Language Models. arXiv preprint arXiv:2209.14876. Preprint; the archival version of this work appeared as PyDex at OOPSLA 2024 (Zhang et al. 2024).

Zhang, J.; Cambronero, J. P.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2024. PyDex: Repairing Bugs in Introductory Python Assignments using LLMs. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1).

Reproducibility Checklist

General paper structure. Conceptual outline of the approach: yes (Algorithm 1, Figure 1). Claims vs. speculation delineated: yes (RQ answer boxes state measured claims; the closing recipe is marked as argument). Pedagogical references for background: yes (Related Work).

Theoretical contributions. This paper makes no formal theoretical claims; the break-even expression in RQ2 is elementary algebra over measured quantities, with all symbols defined in the text.

Datasets. Novel benchmark: yes; motivation (contamination avoidance) and construction are described in Experimental Setup, with generation seeded and scripted. All benchmark artifacts (assignments, tests, reference solutions, buggy programs, splits) are released in the supplementary repository. No proprietary or human-subject data are used.

Computational experiments. Source code, prompts, synthesized repairers, raw LLM responses, and analysis scripts are released; experiments run on a single CPU container (no GPU). Model version, pricing snapshot, tokenizer convention, decoding budget (single sample), seeds for benchmark generation and splits, timeouts, and the RPS threshold are stated in Experimental Setup and are fixed in the released configuration. Metrics are formally defined; repair rates carry 95% Wilson intervals and paired claims use exact McNemar tests. Local search and evaluation are deterministic; LLM calls are single-shot by design (deployment matches one attempt per submission), so no cross-run variance is reported for them—a limitation noted in Discussion.